Introduction to Programming in C++
Seventh Edition

Chapter 7

Objectives

- Differentiate between a pretest loop and a posttest loop
- Include a pretest loop in pseudocode
- Include a pretest loop in a flowchart
- Code a pretest loop using the C++ `while` statement
- Utilize counter and accumulator variables
- Code a pretest loop using the C++ `for` statement

Repeating Program Instructions

- The **repetition structure**, or **loop**, processes one or more instructions repeatedly
- Every loop contains a Boolean condition that controls whether the instructions are repeated
- A **looping condition** says whether to continue looping through instructions
- A **loop exit condition** says whether to stop looping through the instructions
- Every looping condition can be expressed as a loop exit condition (its opposite)

Repeating Program Instructions (cont'd.)

- C++ uses looping conditions in repetition structures
- A repetition structure can be pretest or posttest
- In a **pretest loop**, the condition is evaluated *before* the instructions in the loop are processed
- In a **posttest loop**, the condition is evaluated *after* the instructions in the loop are processed
- In both cases, the condition is evaluated with each repetition

Repeating Program Instructions (cont'd.)

- The instructions in a posttest loop will always be processed at least once
- Instructions in a pretest loop may not be processed if the condition initially evaluates to false
- The repeatable instructions in a loop are called the **loop body**
- The condition in a loop must evaluate to a Boolean value

Repeating Program Instructions (cont'd.)

Problem specification

A superheroine named Isis must prevent a poisonous yellow spider from attacking King Khafra and Queen Rashida. Isis has one weapon at her disposal: a laser beam that shoots out from her right hand. Unfortunately, Isis gets only one shot at the spider, which is flying around the palace looking for the king and queen. Before taking the shot, she needs to position both her right arm and her right hand toward the spider. After taking the shot, she should return her right arm and right hand to their original positions. In addition, she should say "You are safe now. The spider is dead." if the laser beam hit the spider; otherwise, she should say "Run for your lives, my king and queen!"

```
1. position both your right arm and your right hand toward the spider
2. shoot a laser beam at the spider
3. return your right arm and right hand to their original positions
4. if (the laser beam hit the spider)
        say "You are safe now. The spider is dead."
   else
        say "Run for your lives, my king and queen!"
   end if
```

Figure 7-1 A problem that requires the sequence and selection structures

Repeating Program Instructions (cont'd.)

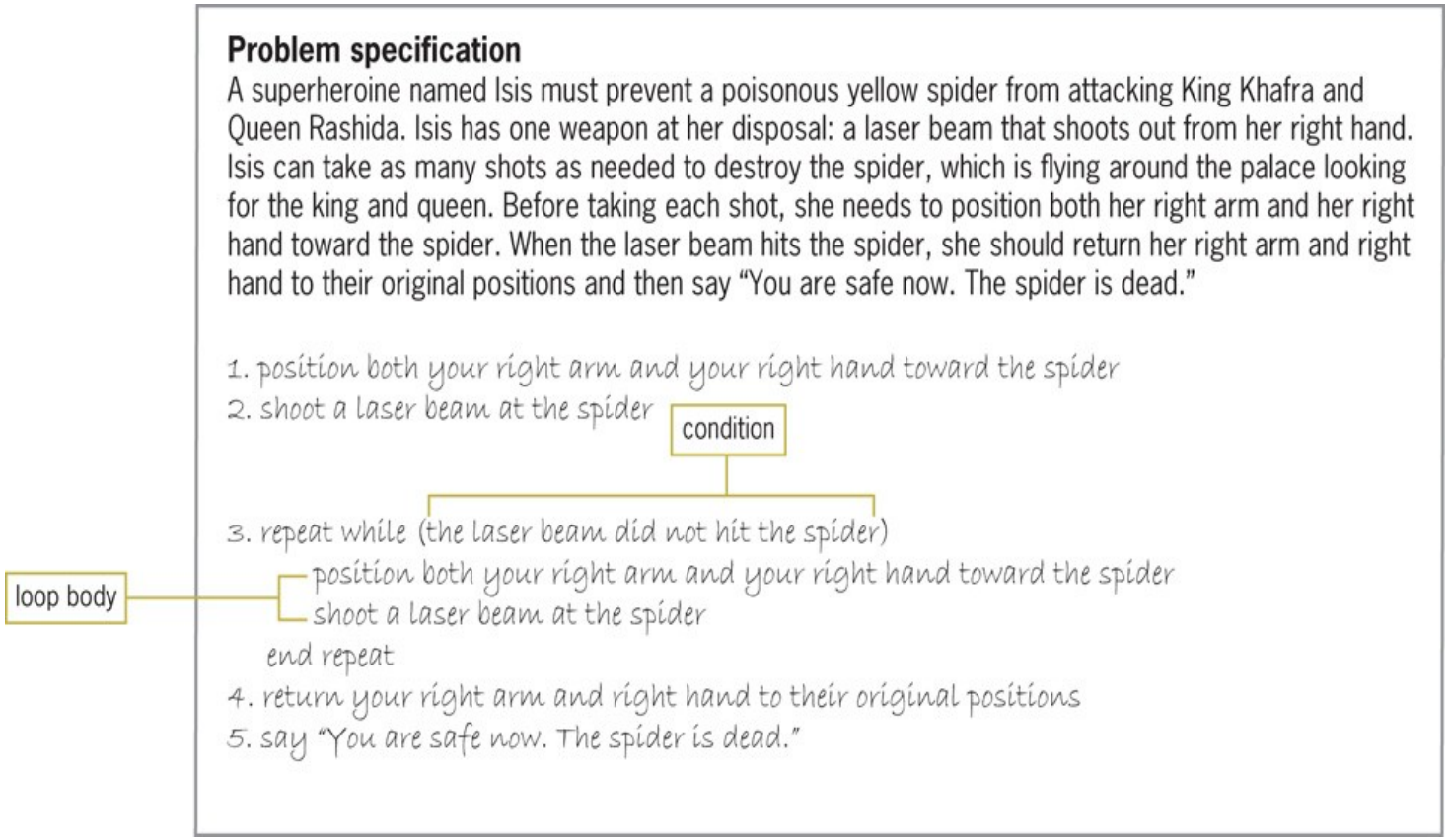


Figure 7-2 A problem that requires the sequence and repetition structures

Using a Pretest Loop to Solve a Real-World Problem

- Most loops have a condition and a loop body
- Some loops require the user to enter a special **sentinel value** to end the loop
- Sentinel values should be easily distinguishable from the valid data recognized by the program
- When a loop's condition evaluates to true, the instructions in the loop body are processed
- Otherwise, the instructions are skipped and processing continues with the first instruction after the loop

Using a Pretest Loop to Solve a Real-World Problem (cont.)

- After each processing of the loop body (iteration), the loop's condition is reevaluated to determine if the instructions should be processed again
- A **priming read** is an instruction that appears before a loop and is used to set up the loop with an initial value entered by the user
- An **update read** is an instruction that appears within a loop that allows the user to enter a new value at each iteration of the loop

Using a Pretest Loop to Solve a Real-World Problem (cont'd.)

Problem specification

Create a program that calculates the commission for each of a company's salespeople. The commission is calculated by multiplying the salesperson's sales amount by 20%.

Input	Processing	Output
commission rate (20%) sales	Processing items: none	commission

calculates and displays the commission for one salesperson only

```
Algorithm 1 (without a loop):
1. enter the sales
2. calculate the commission by multiplying
    the sales by the commission rate
3. display the commission
```

calculates and displays the commission for as many salespeople as needed

```
Algorithm 2 (with a loop):
1. enter the sales
2. repeat while (the sales are not equal to -1)
        calculate the commission by multiplying
        the sales by the commission rate
        display the commission
        enter the sales
   end repeat
```

Figure 7-3 Problem specification and IPO chart for the commission program

Using a Pretest Loop to Solve a Real-World Problem (cont'd.)

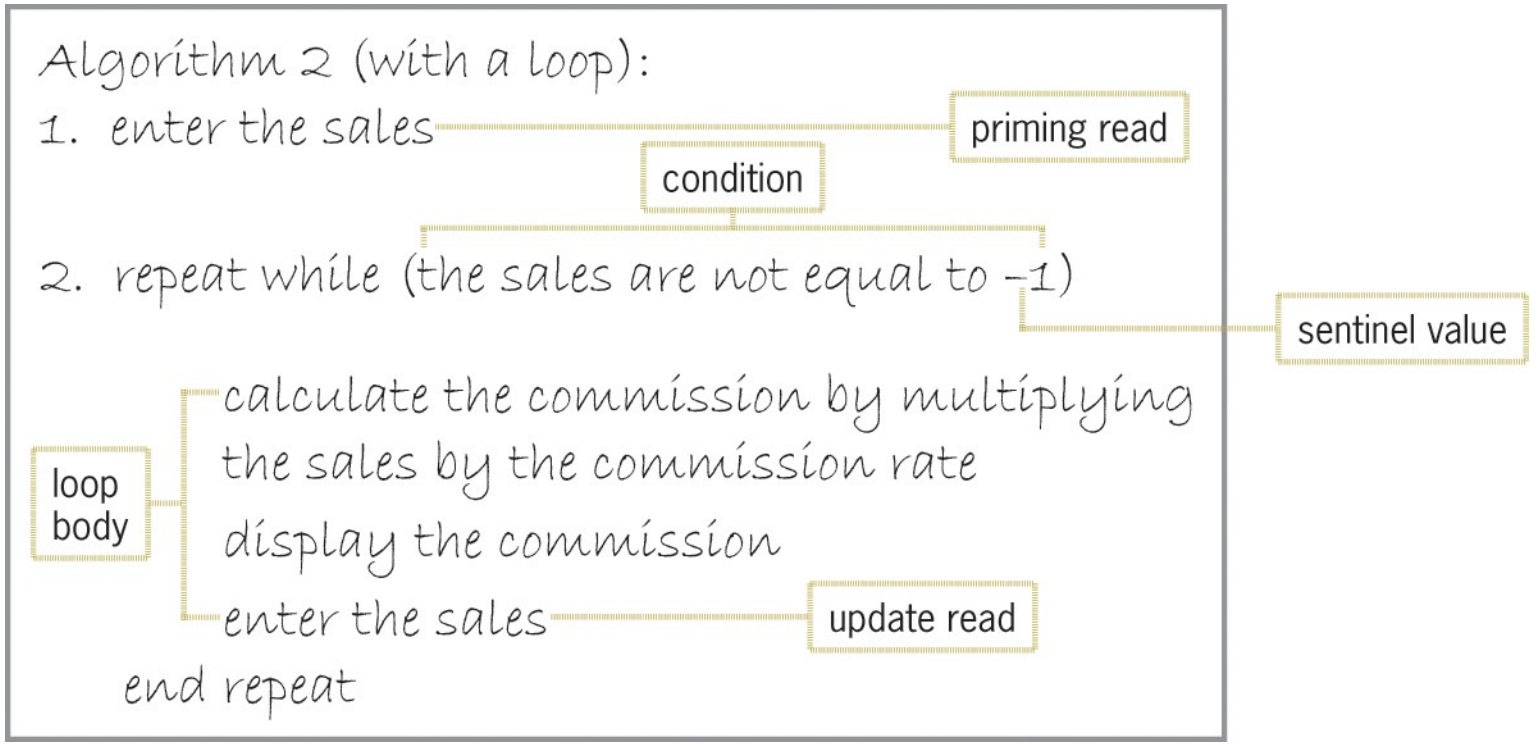


Figure 7-4 Components of Algorithm 2 from Figure 7-3

Flowcharting a Pretest Loop

- The diamond symbol in a flowchart is the decision symbol – represents repetition structures
- A diamond representing a repetition structure contains a Boolean condition
- The condition determines whether the instructions in the loop are processed
- A diamond representing a repetition structure has one flowline leading into it and two leading out

Flowcharting a Pretest Loop (cont'd.)

- The flowlines leading out are marked "T" (true) and "F" (false)
- The "T" line points to the loop body
- The "F" line points to the instructions to be processed if the loop's condition evaluates to false
- The flowline entering the diamond and symbols and flowlines of the true path form a circle, or loop
- This distinguishes a repetition structure from a selection structure in a flowchart

Flowcharting a Pretest Loop (cont'd.)

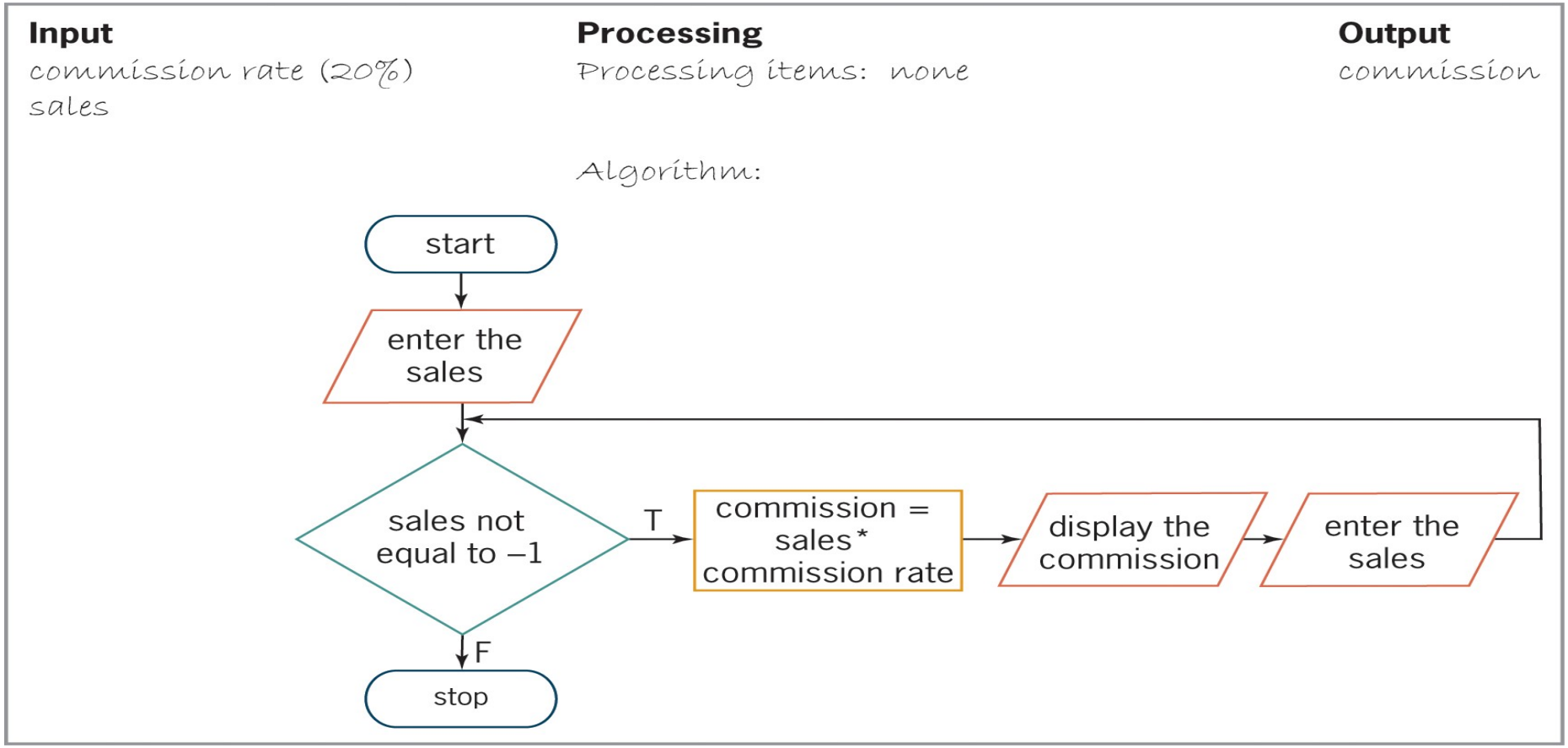


Figure 7-5 Flowchart for algorithm shown in Figure 7-4

Flowcharting a Pretest Loop (cont'd.)

Figure 7-6 First sales entry recorded in the desk-check table

Figure 7-7 First salesperson's entry in the desk-check table

Flowcharting a Pretest Loop (cont'd.)

Figure 7-8 Second salesperson's entry in the desk-check table

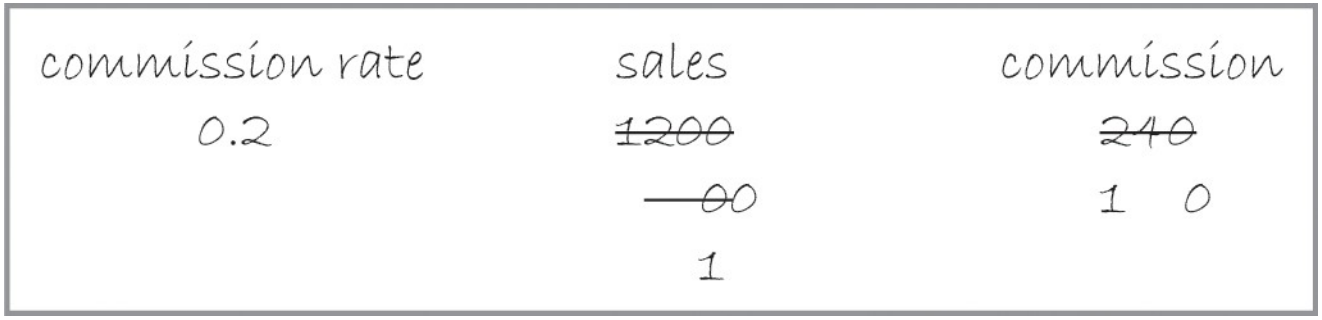


Figure 7-9 Completed desk-check table

The `while` Statement

- You can use the `while` statement to code a pretest loop in C++
- Syntax is:

  `while` (*condition*)

  one statement or a statement block to be processed as long as the condition is true
- Must supply looping condition (Boolean expression)
- *condition* can contain constants, variables, functions, arithmetic operators, comparison operators, and logical operators

The `while` Statement (cont'd.)

- Must also provide loop body statements, which are processed repeatedly as long as condition is true
- If more than one statement in loop body, must be entered as a statement block (enclosed in braces)
- Can include braces even if there is only one statement in the statement block
- Good programming practice to include a comment, such as `//end while`, to mark the end of the `while` statement

The `while` Statement (cont'd.)

- A loop whose instructions are processed indefinitely is called an **infinite loop** or **endless loop**
- You can usually stop a program that has entered an infinite loop by pressing Ctrl+c

The `while` Statement (cont'd.)

HOW TO Use the while Statement

Syntax

while (*condition*)
    either one statement or a statement block to be processed as long as the condition is true
 //end while

Example 1

```
int age = 0;

cout << "Enter age: ";
cin >> age;
while (age > 0)
{
      cout << "You entered " << age << endl;
      cout << "Enter age: ";
      cin >> age;
}     //end while
```

Example 2

```
char anotherSale = ' ';
double sales = 0.0;

cout << "Enter a sales amount? (Y/N) ";
cin >> anotherSale;
while (toupper(anotherSale) == 'Y')
{
      cout << "Enter the sales: ";
      cin >> sales;
      cout << "You entered " << sales << endl;
      cout << "Enter a sales amount? (Y/N) ";
      cin >> anotherSale;
}     //end while
```

Note: You could also write the `while` clause in Example 2 as either `while (tolower(anotherSale) == 'y')` or `while (anotherSale == 'Y' || anotherSale == 'y')`.

Figure 7-10 How to use the `while` statement

The `while` Statement (cont'd.)

IPO chart information

Input

commission rate (20%)
sales

Processing

none

Output

commission

Algorithm

enter t e sales

2 re eat ile (t e sales are not e al to
calc late t e commission m lti l in
t e sales t e commission rate
is la t e commission

enter t e sales

en re eat

C++ instructions

```
const double RATE = 0.2;
int sales = 0;

double commission = 0.0;

cout << "First salesperson's
sales (-1 to stop): ";
cin >> sales;

while (sales != -1)
{
commission = sales * RATE;

    cout << "Commission: $"
    << commission << endl << endl;
    cout << "Next salesperson's
    sales (-1 to stop): ";
    cin >> sales;
}   //end while
```

Figure 7-11 IPO chart information and C++ instructions for the commission program

The `while` Statement (cont'd.)

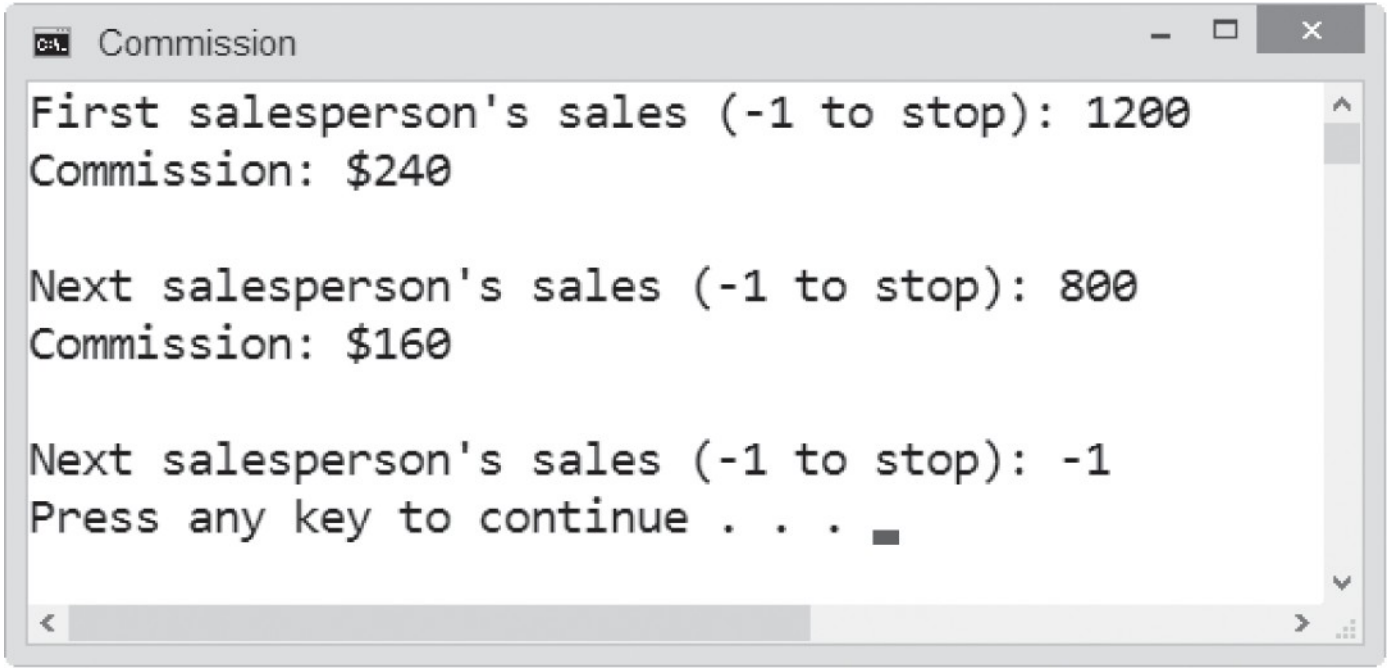


Figure 7-12 A sample run of the commission program

Using Counters and Accumulators

- Some problems require you to calculate a total or average
- To do this, you use a counter, accumulator, or both
- A **counter** is a numeric variable used for counting something
- An **accumulator** is a numeric variable used for accumulating (adding together) multiple values
- Two tasks are associated with counters and accumulators: initializing and updating

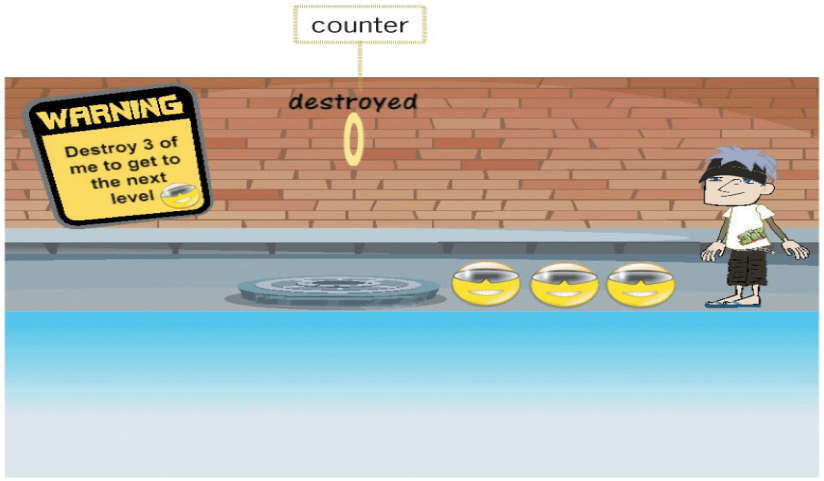
Using Counters and Accumulators (cont'd.)

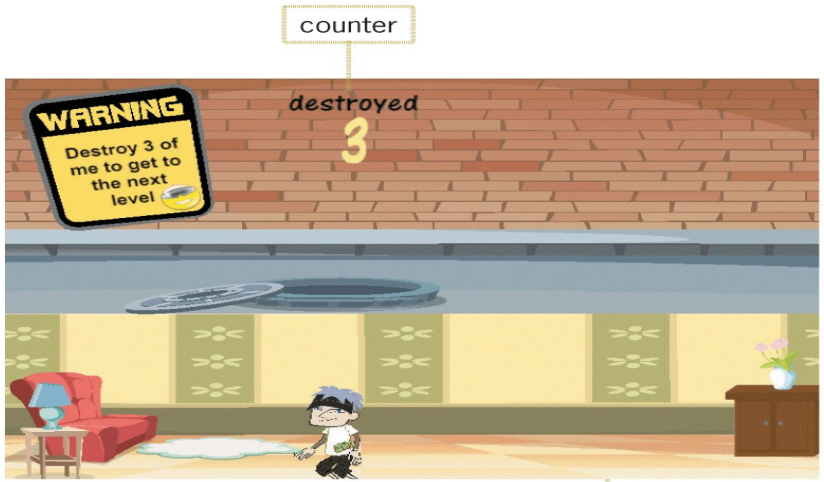
- **Initializing** means assigning a beginning value to a counter or accumulator (usually 0) – happens once, before the loop is processed
- **Updating** (or **incrementing**) means adding a number to the value of a counter or accumulator
- A counter is updated by a constant value (usually 1)
- An accumulator is updated by a value that varies
- Update statements are placed in the body of a loop since they must be performed at each iteration

Using Counters and Accumulators (cont'd.)

Problem specification

To advance to the next level in the game, Eddie must destroy the three smiley faces by jumping on each one. He then must jump through the manhole.






```
1. initialize destroyed counter to 0
2. repeat while (destroyed counter is less than 3)
       jump on smiley face to destroy it
       add 1 to destroyed counter
   end repeat
3. jump into manhole to advance to the next level
```

initialization task

update task

next level

Figure 7-13 Example of a partial game program that uses a counter

Using Counters and Accumulators (cont'd.)

HOW TO Update Counters and Accumulators

Syntax

counterVariable = counterVariable {+ | –} constantValue;
counterVariable {+= | –=} constantValue;

accumulatorVariable = accumulatorVariable {+ | –} value;
accumulatorVariable {+= | –=} value;

Counter examples

```
years = years + 1;
years += 1;
evenNum = evenNum - 2;
evenNum -= 2;
```

Accumulator examples

```
sum = sum + num;
sum += num;
total = total + score;
total += score;
```

Figure 7-14 Syntax and examples of update statements for counters and accumulators

The Stock Price Program

- Example problem and program solution (following slides)
  - Program takes in a sequence of stock prices from the keyboard and outputs their average
  - Uses a counter to keep track of the number of prices entered and an accumulator to keep track of the total
  - Both are initialized to 0
  - The loop ends when the user enters a sentinel value (-1)

The Stock Price Program (cont'd.)

Problem specification

Create a program that allows the user to enter the closing price of a specific stock for any number of days. Use a negative number as the sentinel value. If the sentinel value is the first price the user enters, display the "No stock prices entered" message on the screen. Otherwise, use a counter to keep track of the number of prices entered and an accumulator to total the prices. When the user has finished entering the prices, calculate the average price by dividing the accumulator's value by the counter's value, and then display the average price on the screen.

Figure 7-15 Problem specification for the Sales Express program

The Stock Price Program (cont'd.)

IPO chart information

Input

price

Processing

er price c er
price cc r

Output

er e price

Algorithm

e er e price

repe i e e price i e

e er price

e price e price

e er e price

e repe
i e er price i re

c c e e er e price
i i i e price
er price

i p e e er e pric

e e
i p c price e ere e

e i

C++ instructions

```
double price = 0.0;

int numPrices = 0;
double totalPrices = 0.0;

double avgPrice = 0.0;

cout << "Closing price (negative
number to stop): ";
cin >> price;

while (price >= 0.0)
{
     numPrices += 1;;

     totalPrices += price;

     cout << "Closing price
     (negative number to stop): ";
     cin >> price;
}    //end while
if (numPrices > 0)
{    er
     avgPrice = totalPrices /
     numPrices;
        e

     cout << "Average price: $"
     << avgPrice << endl;
}
else
     cout << "No stock prices
     entered" << endl;
//end if
```

Figure 7-15 IPO chart information and C++ instructions for the Stock Price program

The Stock Price Program (cont'd.)

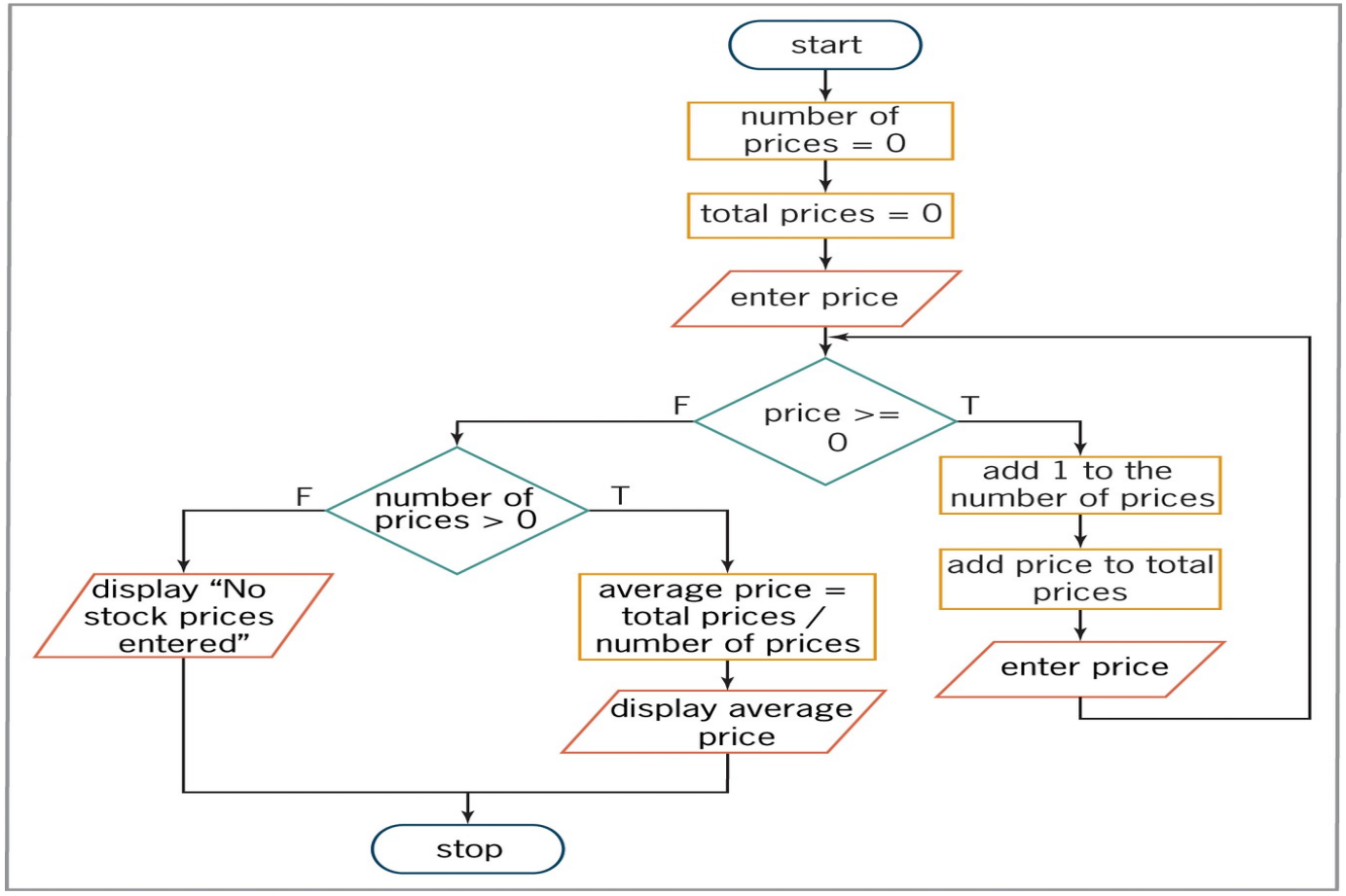


Figure 7-16 Flowchart for the Stock Price program

The Stock Price Program (cont'd.)

Figure 7-17 Desk-check table after the first update to the counter and accumulator variables

Figure 7-18 Desk-check table after the second update to the counter and accumulator variables

The Stock Price Program (cont'd.)

Figure 7-19 Completed desk-check table and sample runs for the Stock Price program

Counter-Controlled Pretest Loops

- Loops can be controlled using a counter rather than a sentinel value
- These are called **counter-controlled loops**
- Example problem and program solution (following slides)
  - Counter-controlled loop is used that totals the quarterly sales from three regions
  - Loop repeats three times, once for each region, using a counter to keep track

Counter-Controlled Pretest Loops (cont'd.)

Modified problem specification (original shown in Figure 7-15)
Create a program that allows the user to enter the closing price of a specific stock for each of five days. Use a counter to keep track of the number of days and an accumulator to total the five prices. Calculate the average price by dividing the accumulator's value by a number that is one less than the counter's value, and then display the average price on the screen.

IPO chart information

Input

price

Processing

er c
price cc

Output

er e price

Algorithm

repe i e e
e
e er e price

C++ instructions

```
double price = 0.0;

int numDays = 1;
double totalPrices = 0.0;

double avgPrice = 0.0;

while (numDays < 6)
{
    cout << "Day " << numDays <<
    " closing price: ";
    cin >> price;
```

Figure 7-20 Problem specification, IPO chart information, and C++ instructions for the modified Stock Price program

Counter-Controlled Pretest Loops (cont'd.)

```
                                    numDays += 1;;

                                    totalPrices += price;
                                }   //end while

                                avgPrice = totalPrices / (numDays - 1);

                                    cout << "Average price: $" << avgPrice <<
                                    endl;
```

Figure 7-20 Problem specification, IPO chart information, and C++ instructions for the modified Stock Price program (cont'd.)

Counter-Controlled Pretest Loops (cont'd.)

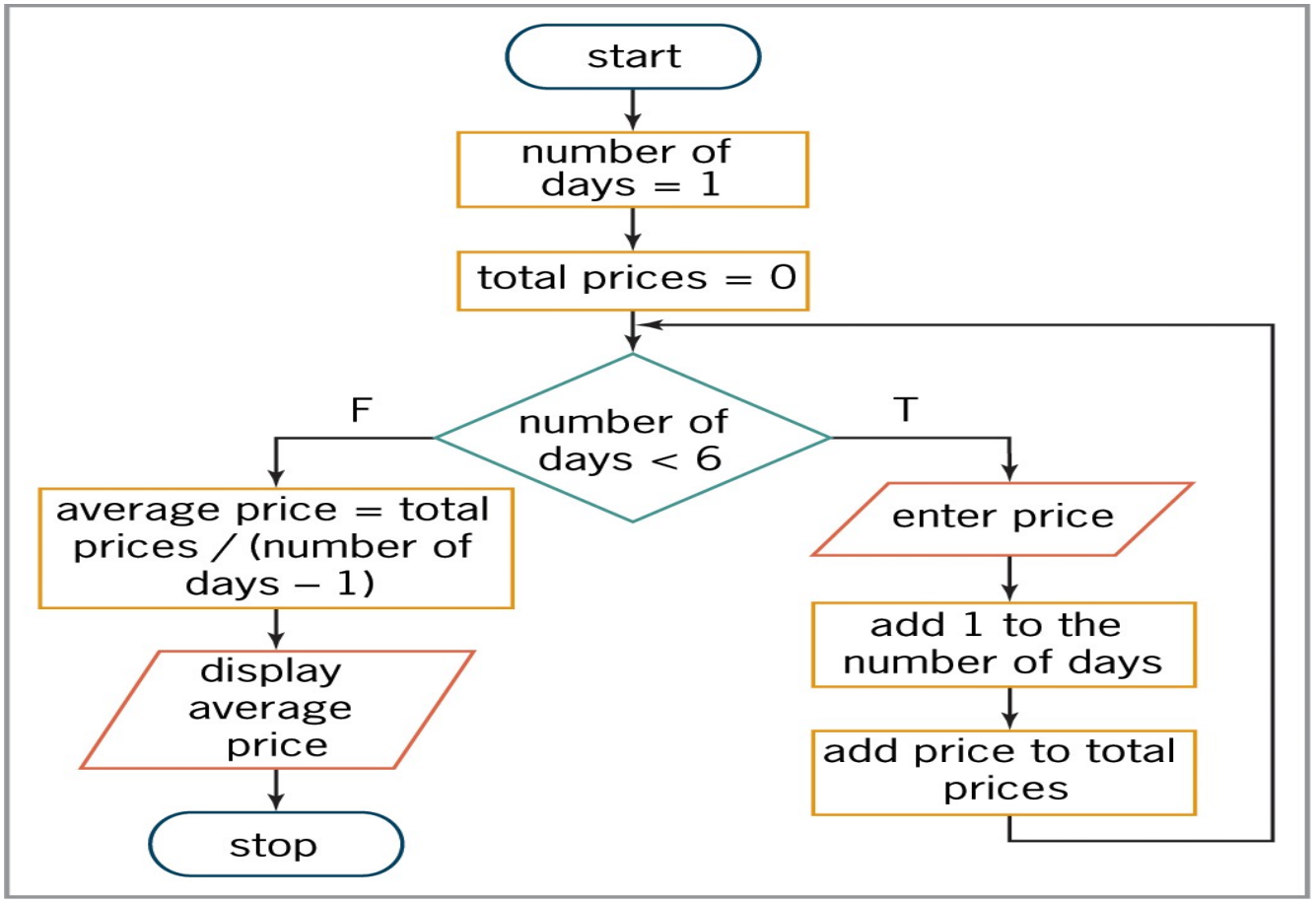


Figure 7-21 Flowchart for the modified Stock Price program

Counter-Controlled Pretest Loops (cont'd.)

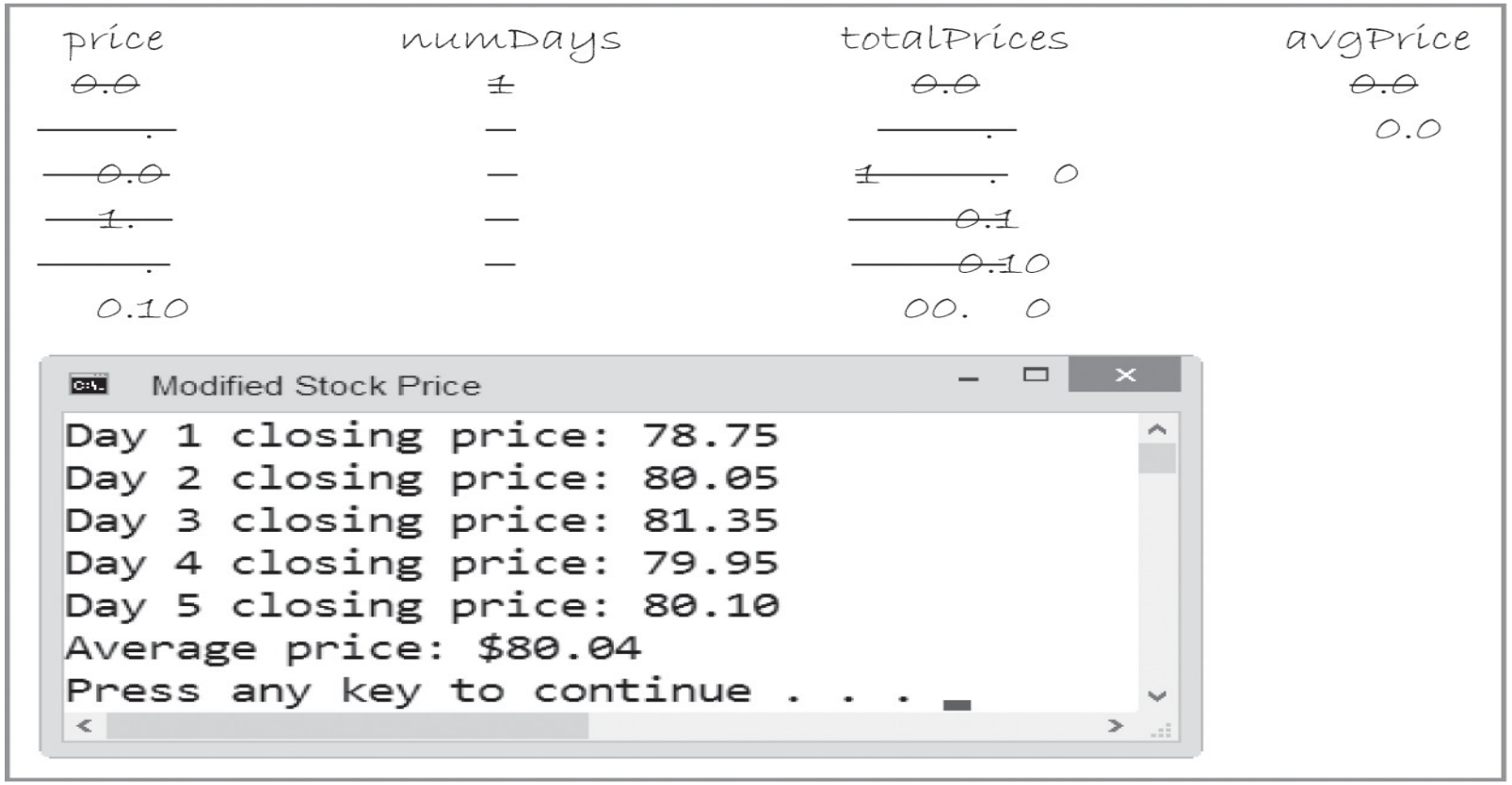


Figure 7-22 Completed desk-check table and sample run for the modified Stock Price program

The `for` Statement

- The `for` statement can also be used to code any pretest loop in C++
- Commonly used to code counter-controlled pretest loops (more compact than `while` in this case)
- Syntax:

`for` ([*initialization*]; *condition;* [*update*])

one statement or a statement block to be processed as long as condition is true

- *initialization* and *update* arguments are optional

The `for` Statement (cont'd.)

- *initialization* argument usually creates and initializes a counter variable
- Counter variable is local to `for` statement (can only be used inside the loop)
- *condition* argument specifies condition that must be true for the loop body to be processed
- *condition* must be a Boolean expression
  - May contain variables, constants, functions, arithmetic operators, comparison operators, and logical operators

The `for` Statement (cont'd.)

- Loop ends when *condition* evaluates to false
- *update* argument usually contains an expression that updates the counter variable
- Loop body follows the `for` clause
  - Must be placed in braces if it contains more than one statement
  - Braces can be used even if the loop body contains only one statement
- Good programming practice to place a comment, such as `//end for`, to mark the end of a `for` loop

The `for` Statement (cont'd.)

HOW TO Use the for Statement

Syntax

semicolons

for ([*initialization*]; *condition*; [*update*])
    either one statement or a statement block to be processed as long as the condition is true
//end for

Example 1: displays the numbers 1, 2, and 3 on separate lines on the screen

```
for (int x = 1; x < 4; x += 1)
    cout << x << endl;
//end for
```

Example 2: displays the numbers 3, 2, and 1 on separate lines on the screen

```
for (int x = 3; x > 0; x -= 1)
{
    cout << x << endl;
}  //end for
```

Note: The *condition* and *update* arguments in Example 1 can also be phrased as `x <= 3` and `x = x + 1`, respectively. In Example 2, the *condition* and *update* arguments can also be phrased as `x >= 1` and `x = x - 1`, respectively.

Figure 7-23 How to use the `for` statement

The `for` Statement (cont'd.)

Example 1: displays the numbers 1, 2, and 3 on separate lines on the screen

```
for (int x = 1; x < 4; x += 1)
    cout << x << endl;
//end for
```

Processing steps

1. The *initialization* argument (`int x = 1`) creates a variable named `x` and initializes it to 1.
2. The *condition* argument (`x < 4`) checks whether the `x` variable's value is less than 4. It is, so the statement in the loop body displays the `x` variable's value (1) on the screen.
3. The *update* argument (`x += 1`) adds 1 to the contents of the `x` variable, giving 2.
4. The *condition* argument checks whether the `x` variable's value is less than 4. It is, so the statement in the loop body displays the `x` variable's value (2) on the screen.
5. The *update* argument adds `1` to the contents of the `x` variable, giving 3.
6. The *condition* argument checks whether the `x` variable's value is less than 4. It is, so the statement in the loop body displays the `x` variable's value (3) on the screen.
7. The *update* argument adds 1 to the contents of the `x` variable, giving 4.
8. The *condition* argument checks whether the `x` variable's value is less than 4. It's not, so the `for` loop ends. Processing continues with the statement following the end of the loop.

Figure 7-24 Processing steps for Example 1's code

The Total Payroll Program

- Extended example of a problem and program solution (following slides)
  - Program totals up the payroll from three stores using a `for` loop

The Total Payroll Program (cont'd.)

Problem specification
Create a program that allows the user to enter the payroll amount for each of a company's three stores. The program should calculate the total payroll and then display the result on the screen. Use a counter to ensure that the user enters exactly three payroll amounts. Use an accumulator to total the amounts.

IPO chart information

Input
store's payroll

Processing
er o stores o ter to

Output
total payroll a lator

Algorithm
repeat or er o stores ro to
e ter t e store s payroll
a t e store s payroll to t e total payrol
e repeat
splay t e total payroll

C++ instructions

```
int storePayroll = 0;

t s ar a le s reate a
t e or la se

int totalPayroll = 0;

for (int numStores = 1; numStores
<= 3; numStores += 1)
{
    cout << "Store " << numStores
    << " payroll: ";
    cin >> storePayroll;
    totalPayroll += storePayroll;
}   //end for
cout << "Total payroll: $" <<
totalPayroll << endl;
```

Figure 7-25 Program specification, IPO chart information, and C++ instructions for the Total Payroll program

The Total Payroll Program (cont'd.)

Figure 7-26 Results of processing the declaration statements and *initialization* argument

Figure 7-27 Desk-check table after *update* argument is processed first time

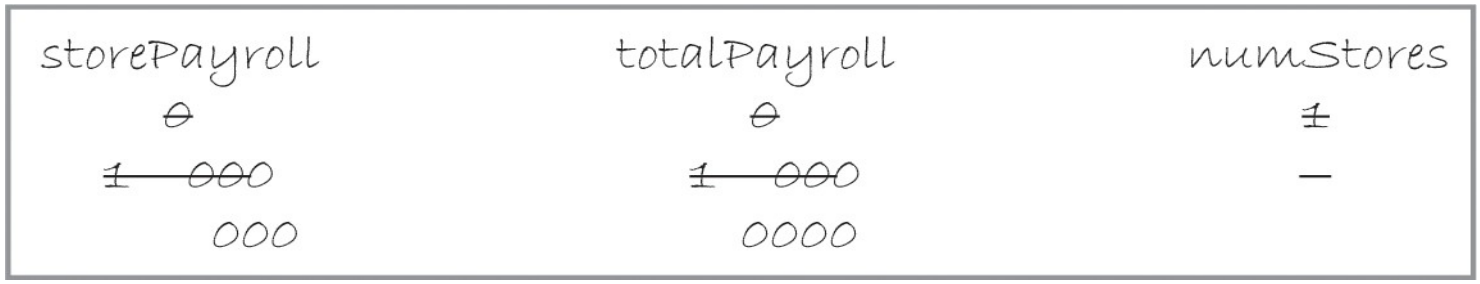


Figure 7-28 Desk-check table after *update* argument is processed second time

The Total Payroll Program (cont'd.)

storePayroll	totalPayroll	numStores
0	0	1
1 000	1 000	2
000	0000	3
0000	0000	

Figure 7-29 Desk-check table after *update* argument is processed third time

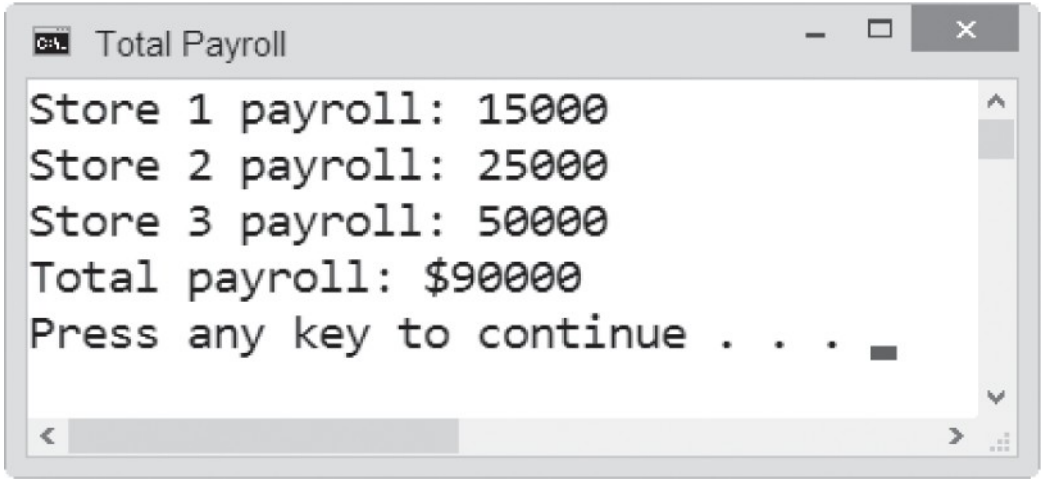


Figure 7-30 Sample run of the Total Payroll program

The Tip Program

- Extended example of a problem and program solution (following slides)
  - Calculates the ti for a given sales amount using three different rates
  - A `for` loop keeps track of each of the three rates

The Tip Program (cont'd.)

Problem specification
Create a program that allows the user to enter the amount of a restaurant bill. The program should calculate and display the suggested amounts to tip a waiter using rates of 10%, 15%, and 20%. Use a counter to keep track of the three rates.

IPO chart information

Input
bill
rate (counter: 10% to 20% in increments of 5%)

Processing
none

Output
ti

Algorithm
1 enter t e bill

2 re eat for (rate from 10% to 20% in increments of 5%)
calculate ti b multi l in t e bill b t e rate

is la t e rate an ti

en re eat

C++ instructions

```
double bill = 0.0;
t is ariable is create an initiali in t e for clause

double tip = 0.0;

cout << "Bill amount: ";
cin >> bill;

for (double rate = 0.1;
rate <= 0.2; rate += 0.05)
{
    tip = bill * rate;

    cout << rate * 100 << "%
    tip: ";
    cout << "$" << tip << endl;
}   //end for
```

Figure 7-31 Problem specification, IPO chart information, and C++ instructions for the Tip program

The Tip Program (cont'd.)

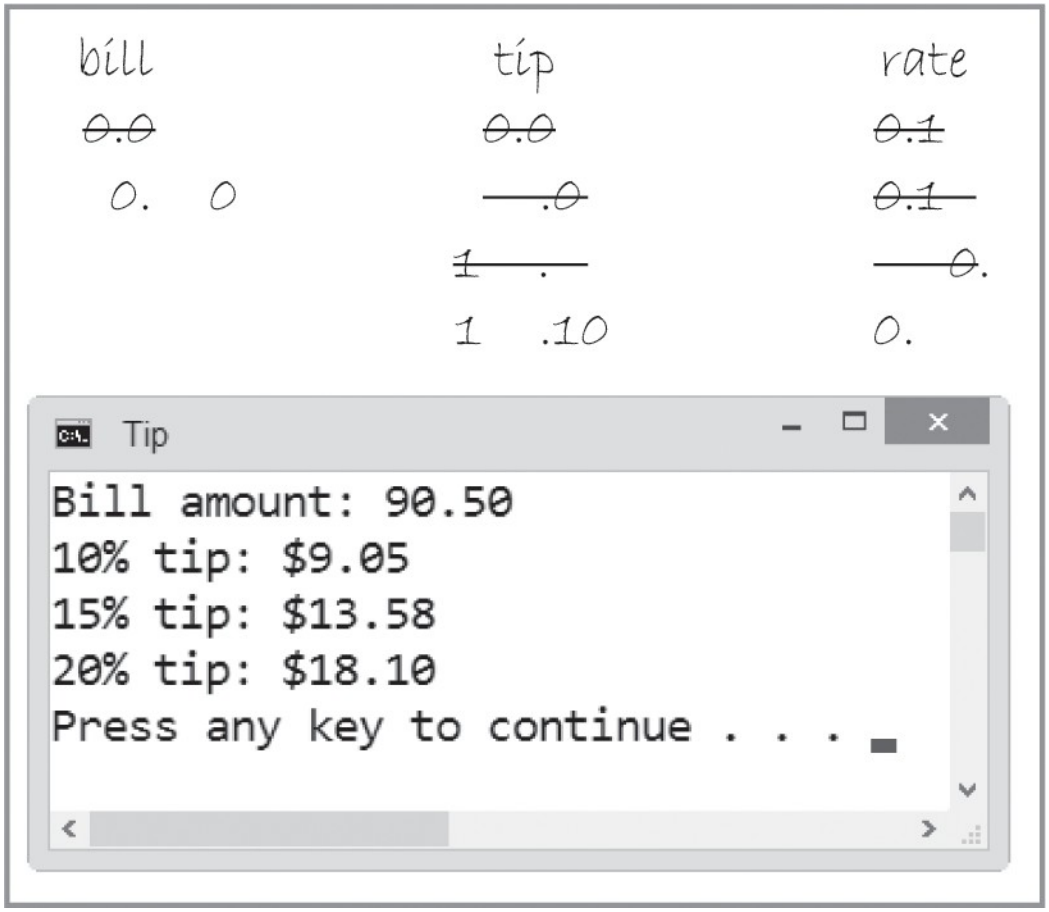


Figure 7-32 Desk-check table and sample run of the Tip program

The Tip Program (cont'd.)

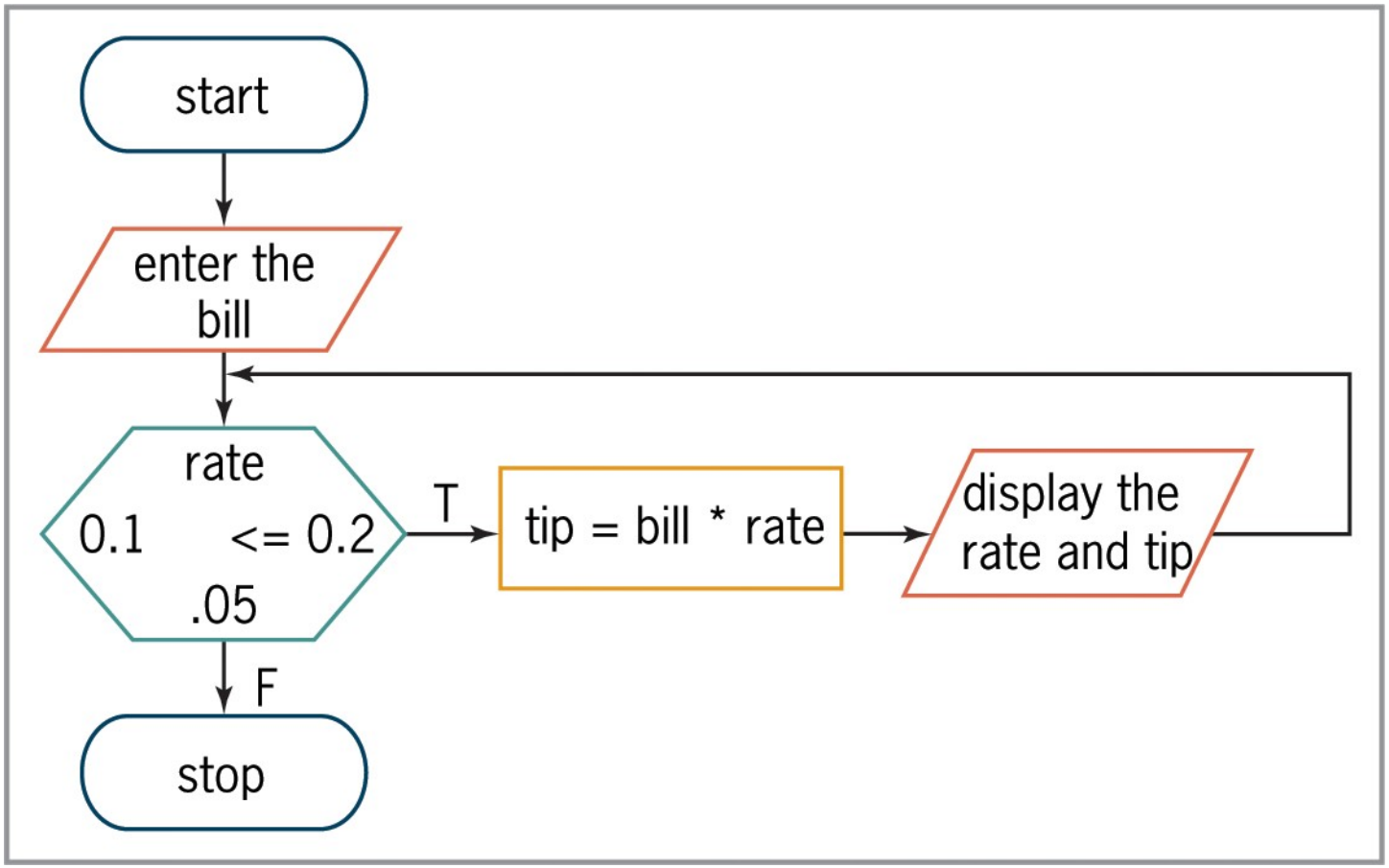


Figure 7-33 Tip program's algorithm shown in flowchart form

Another Version of the Commission Program

- Alternative version of the Commission program (following slides)
  - Uses a `for` loop instead of a `while` loop

Another Version of the Commission Program (cont'd.)

Problem specification
Create a program that calculates the commission for each of a company's salespeople. The commission is calculated by multiplying the salesperson's sales amount by 20%.

IPO chart information

Input
commission rate (20%)
sales

Processing
none

Output
commission

Algorithm
enter t e sales

2 re eat ile (t e sales are not e al to

calc late t e commission m lti l in t e sales t e commission rate

is la t e commission

enter t e sales

en re eat

C++ instructions

```
const double RATE = 0.2;
int sales = 0;

double commission = 0.0;

cout << "First salesperson's
sales (-1 to stop): ";
cin >> sales;
for (; sales != -1;)
{
    commission = sales * RATE;

    cout << "Commission: $"
    << commission << endl << endl;

    cout << "Next salesperson's
    sales (-1 to stop): ";
    cin >> sales;
}   //end for
```

semicolon after the initialization argument
condition argument
semicolon after the condition argument

Figure 7-34 IPO chart information and modified C++ instructions for the Commission program

Another Version of the Commission Program (cont'd.)

Processing steps

1. The `const` statement creates the `RATE` named constant and initializes it to 0.2.
2. The declaration statements create the `sales` and `commission` variables and initialize them to 0 and 0.0, respectively.
3. The first `cout` statement prompts the user to enter the first sales amount, and the `cin` statement stores the user's response (1200) in the `sales` variable.
4. The `for` clause's *condition* argument (`sales != -1`) checks whether the `sales` variable's value is not equal to –1. The condition evaluates to true, so the statements in the loop body

Figure 7-35 Processing steps and sample run for the Commission program shown in Figure 7-34

Another Version of the Commission Program (cont'd.)

calculate and display the commission (240). They also prompt the user for the next sales amount and store the user's response (800) in the `sales` variable.

5. The `for` clause's *condition* argument checks whether the `sales` variable's value is not equal to –1. The condition evaluates to true, so the statements in the loop body calculate and display the commission (160). They also prompt the user for the next sales amount and store the user's response (–1) in the `sales` variable.
6. The `for` clause's *condition* argument checks whether the `sales` variable's value is not equal to –1. In this case, the condition evaluates to false, so the `for` loop ends. Processing continues with the statement following the end of the loop.

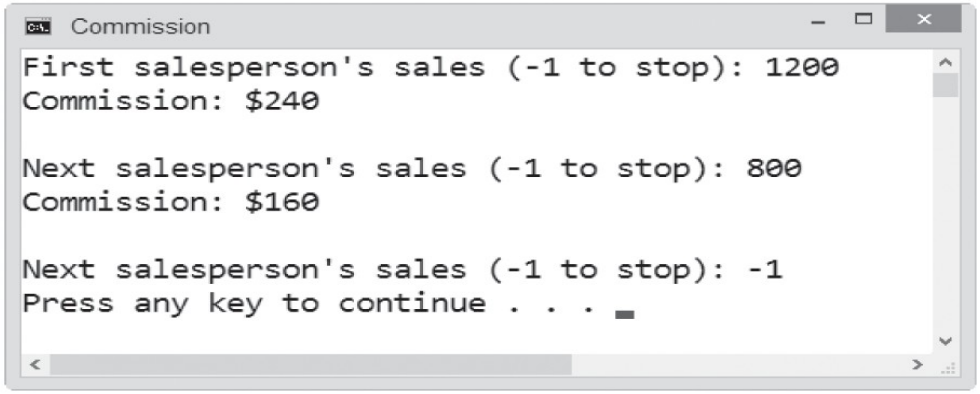


Figure 7-35 Processing steps and sample run for the Commission program shown in Figure 7-34 (cont'd.)

The Even Integers Program

- Program to calculate and display the sum of three integers entered by the user (following slides)
  - Shows incorrect solution via a *for* loop
  - Shows eventual correct solution via *while* loop

The Even Integers Program (cont'd.)

Problem specification

Create a program that calculates and displays the sum of three even integers entered by the user.

Incorrect code using the `for` statement

```
for (int x = 1; x < 4; x += 1)
{
    cout << "Enter an even integer: ";
    cin >> evenInteger;
    if (evenInteger % 2 == 0)
        sum += evenInteger;
    else
        cout << "Please enter an even number." << endl << endl;
   //end if
} //end for
cout << "Sum of the three even integers: " << sum << endl;
```

Figure 7-36 Problem specification and code of the incorrect Even Integers Program

The Even Integers Program (cont'd.)

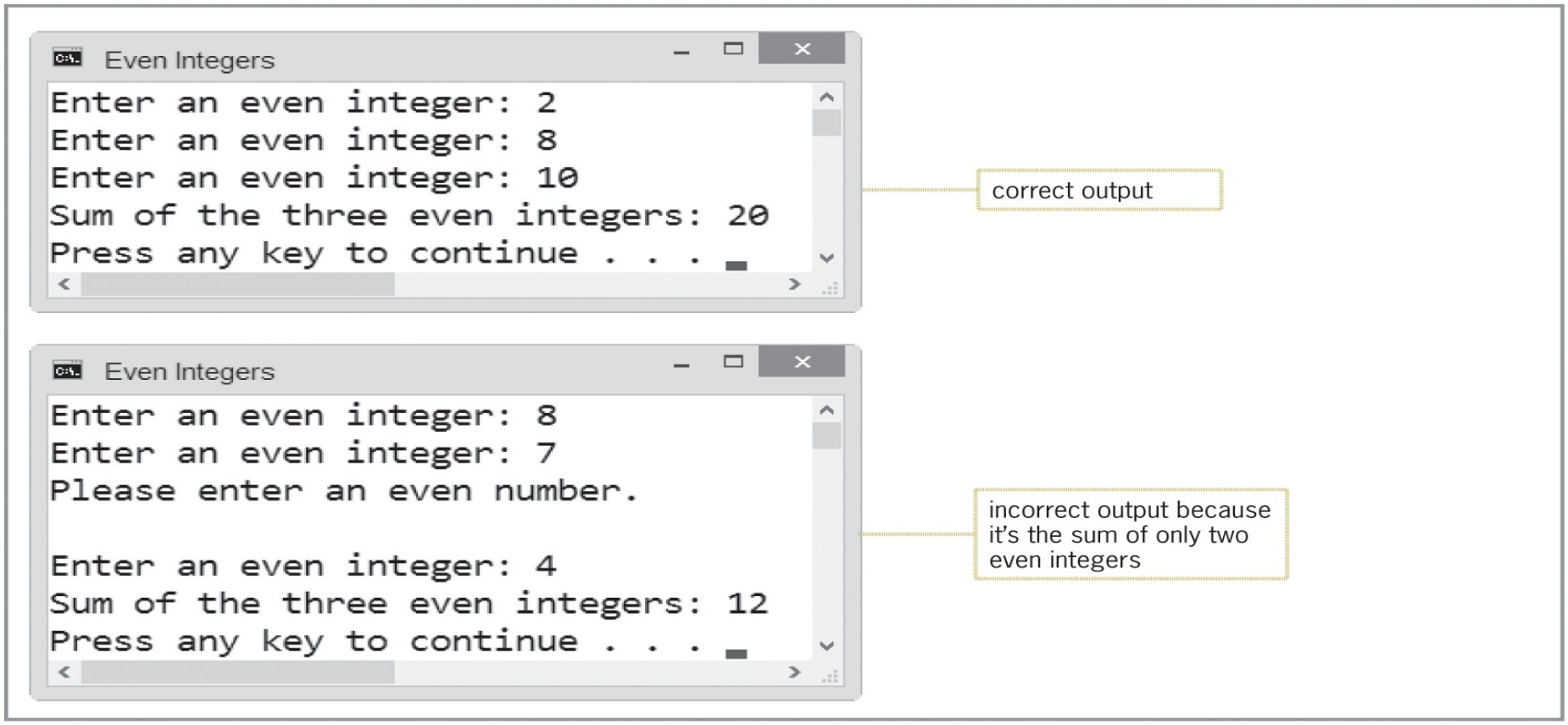


Figure 7-36 Sample runs of the incorrect Even Integers Program

The Even Integers Program (cont'd.)

Incorrect way to fix the program

```
else
{     x -= 1;
      cout << "Please enter an even number." << endl << endl;
}//end if
```

don't modify the value in a for statement's counter variable

Correct way to fix the program

```
int x = 1;
while (x < 4)
{
     cout << "Enter an even integer: ";
     cin >> evenInteger;
     if (evenInteger % 2 == 0)
     {
          sum += evenInteger;
          x += 1;
     }
     else
          cout << "Please enter an even number." << endl << endl;
     //end if
}  //end while
cout << "Sum of the three even integers: " << sum << endl;
```

replace the for statement with the while statement

Figure 7-37 Incorrect and correct ways to fix the Even Integers Program

The Even Integers Program (cont'd.)

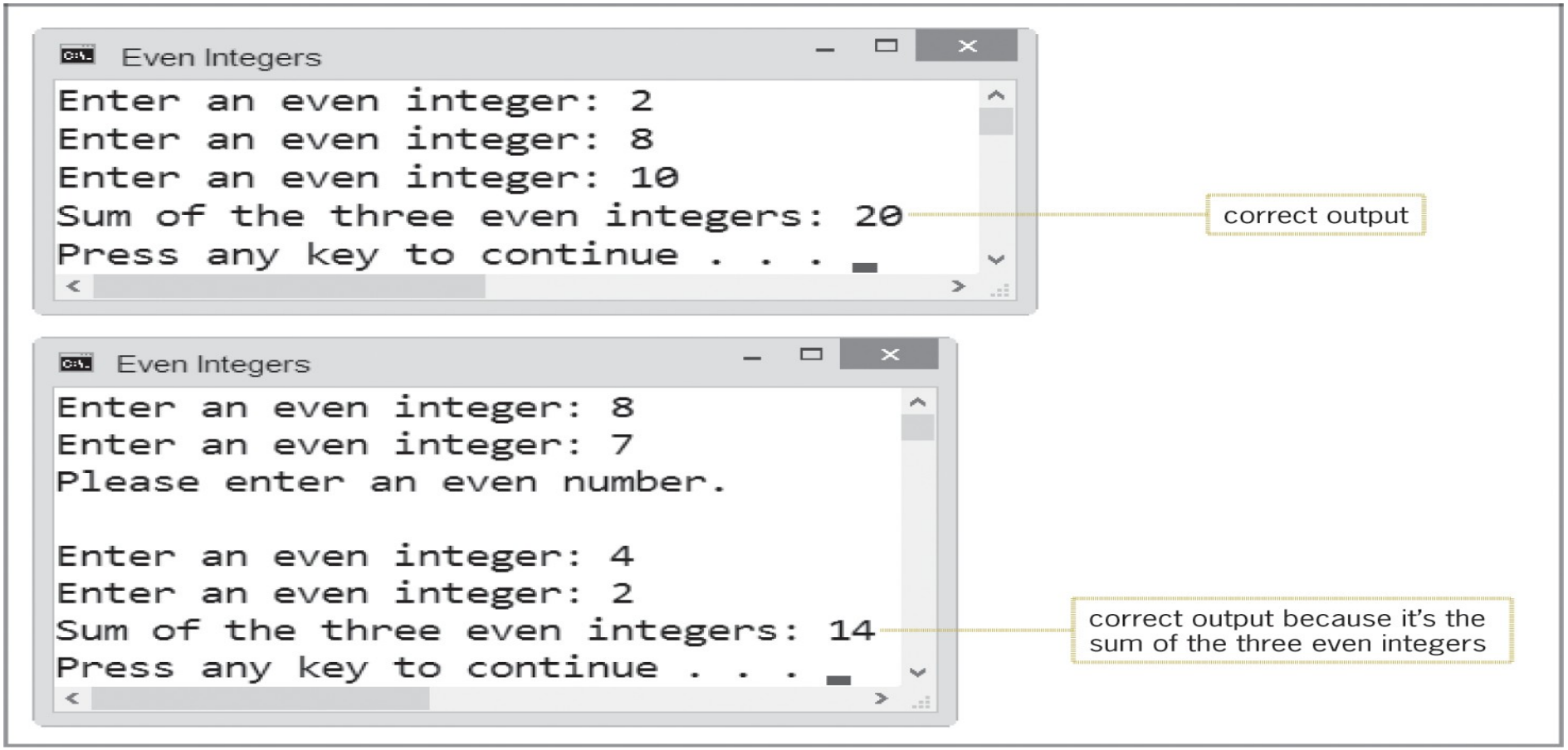


Figure 7-37 Sample runs of the correct solution to the Even Integers Program

Summary

- Use the repetition structure (or loop) to repeatedly process one or more instructions
- Loop repeats as long as looping condition is true (or until loop exit condition has been met)
- A repetition structure can be pretest or posttest
- In a pretest loop, the loop condition is evaluated *before* instructions in loop body are processed
- In a posttest loop, the evaluation occurs *after* instructions in loop body are processed

Summary (cont'd.)

- Condition appears at the beginning of a pretest loop – must be a Boolean expression
- If condition evaluates to true, the instructions in the loop body are processed; otherwise, the loop body instructions are skipped
- Some loops require the user to enter a special sentinel value to end the loop
- Sentinel values should be easily distinguishable from valid data recognized by the program
- Other loops are terminated by using a counter

Summary (cont'd.)

- Input instruction that appears above a pretest loop's condition is the priming read
  - Sets up the loop by getting first value from user
- Input instruction that appears within the loop is the update read
  - Gets the remaining values (if any) from user
- In most flowcharts, diamond (decision symbol) is used to represent a repetition structure's condition

Summary (cont'd.)

- Counters and accumulators are used in repetition structures to calculate totals and averages
- All counters and accumulators must be initialized and updated
- Counters are updated by a constant value
- Accumulators are updated by a variable amount
- You can use either the `while` statement or the `for` statement to code a pretest loop in C++

Lab 7-1: Stop and Analyze

- Study the program in Figure 7-38 on page 229 and answer the questions
- The program calculates the average number of text messages sent each day for 7 days

Lab 7-1: Stop and Analyze (cont'd.)

```
 1 //Lab7-1.cpp - calculates the average number of text
 2 //messages sent each day for 7 days
 3 //Created/revised by <your name> on <current date>
 4
 5 #include <iostream>
 6 #include <iomanip>
 7 using namespace std;
 8
 9 int main()
10 {
11    int day = 0;
12    int totalTexts = 0;
13    int dailyTexts = 0;
14    double average = 0.0;
15
16    for (day = 1; day < 8; day += 1)
17    {
18        cout << "How many text messages did you send on day "
19            << day << "? ";
20        cin >> dailyTexts;
21        totalTexts += dailyTexts;
22    }   //end for
23
24    average = static_cast<double>(totalTexts) / (day - 1);
25    cout << fixed << setprecision(0);
26    cout << endl << "You sent approximately "
27            << average << " text messages per day." << endl;
28    return 0;
29 }  //end of main function
```

Figure 7-38 Code for Lab 7-1

Lab 7-2: Plan and Create

Problem specification

Create a program that displays the number of years required for a company's sales amount to reach at least $150,000, using the current year's sales amount and a 5.5% growth rate per year. The program should also display the sales amount at that time.

Figure 7-39 Problem specification for Lab 7-2

Lab 7-2: Plan and Create (cont'd.)

Input	Processing	Output
growth rate (5.5%) sales	Processing items: annual increase	number of years (counter) sales

Algorithm:

1. enter the sales
2. repeat while (sales < 150000)
   calculate the annual increase by multiplying the sales by the growth rate
   add the annual increase to the sales
   add 1 to the number of years
   end repeat
3. display the number of years and the sales

Figure 7-40 Completed IPO chart for Lab 7-2

Lab 7-2: Plan and Create (cont'd.)

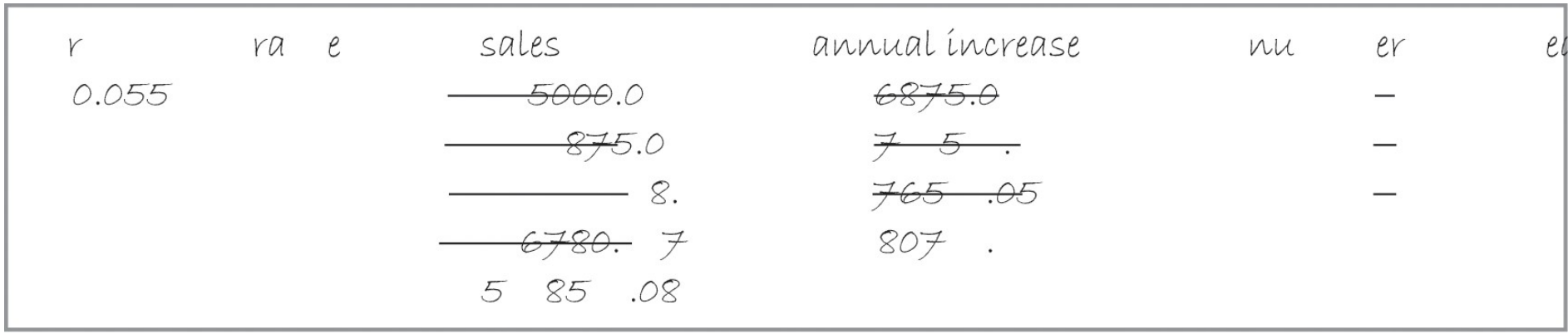


Figure 7-41 Completed desk-check table for Lab 7-2

Lab 7-2: Plan and Create (cont'd.)

IPO chart information

Input

growth rate (5.5%)
sales

Processing

a al rease

Output

er o ears (o ter)
sales

Algorithm

. e ter the sales
re eat wh le (sales 5)

al late the a al rease
lt l g the sales the growth rat

a the a al rease to the sales

a to the er o ears
e re eat
. s la the er o ears a

C++ instructions

```
const double GROWTH_RATE = 0.055;
double sales = 0.0;

double annualIncrease = 0.0;

int years = 0;
  e lare the t se t o

cout << "Current year's sales: ";
cin >> sales;
while (sales < 150000.0)
{
   annualIncrease = sales *
   GROWTH_RATE;
   sales += annualIncrease;

   years += 1;
} // end while
cout << "Sales " << years << " years
from now: $" << sales << endl;
```

Figure 7-42 IPO chart and C++ instructions for Lab 7-2

Lab 7-2: Plan and Create (cont'd.)

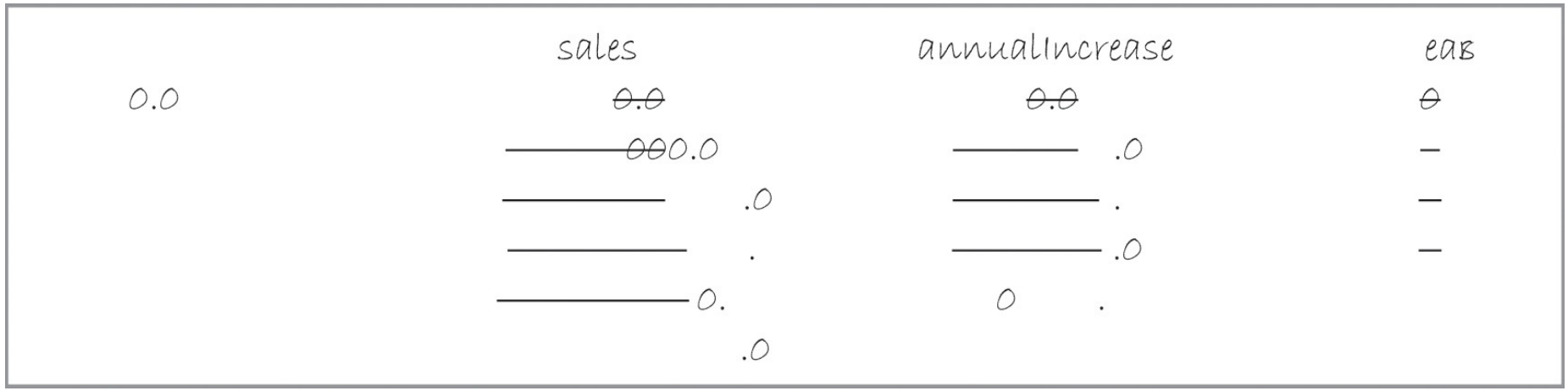


Figure 7-43 Completed desk-check table for Lab 7-2

Lab 7-2: Plan and Create (cont'd.)

```
 1 //Lab7-2.cpp - Displays the number of years required
 2 //for a company's sales to reach at least $150,000
 3 //using a 5.5% annual growth rate. Also displays
 4 //the sales at that time.
 5 //Created/revised by <your name> on <current date>
 6
 7 #include <iostream>
 8 #include <iomanip>
 9 using namespace std;
10
11 int main()
12 {
13    const double GROWTH_RATE = 0.055;
14    double sales = 0.0;
15    double annualIncrease = 0.0;
16    int years = 0;
17
18    cout << "Current year's sales: ";
19    cin >> sales;
20    while (sales < 150000.0)
21    {
22        annualIncrease = sales * GROWTH_RATE;
23        sales += annualIncrease;
24        years += 1;
25    }  //end while
26
27    cout << fixed << setprecision(0);
28    cout << "Sales " << years << " years from now: $"
29        << sales << endl;
30
31    return 0;
32 }  //end of main function
```

Figure 7-44 Finalized program for Lab 7-2

Lab 7-3: Modify

- Modify the program in Lab7-1.cpp to utilize a *while* loop instead of a *for* loop. Save the modified program as Lab7-3.cpp
- Test the program using the following data: 76, 80, 100, 43, 68, 70, and 79.

Lab 7-4: What's Missing?

- The program is this lab should display the average electric bill.
- Follow the instructions for starting C++ and opening the Lab7-4.cpp file.
- Put the C++ instructions in the proper order, and then determine the one or more missing instructions.
- Test the program using the following monthly electric bills: 124.89, 110.65, 99.43, 100.35, and -1 (the sentinel value)

Lab 7-5: Desk-Check

- The code in Figure 7-45 should display the numbers 2, 4, 6, 8, 10, and 12.
- Desk-check the code; did your desk-check reveal any errors?
- If so, correct the code and then desk-check it again

```
for (int number = 2; number < 12; number += 2)
        cout << number << endl;
//end for
```

Figure 7-45 Code for Lab 7-5

Lab 7-6: Debug

- Follow the instructions for starting C++ and opening the Lab7-6.cpp file
- Run the program and enter 15.45 when prompted
- The program goes into an infinite loop
- Type Ctrl+c to end the program
- Debug the program